

Tasks

1. Networking – Python TCP/UDP Communication

- Implement a Python **TCP Echo Server** that accepts multiple client connections.
- Implement a Python **Client** that can connect to the server, send messages, and receive responses.
- Add support for **UDP** as well (server should handle both TCP and UDP messages).

Acceptance Criteria:

- Server runs on configurable host/port.
 - Multiple clients can connect concurrently.
 - Both TCP and UDP messages are processed.
 - Server logs all connections and messages.
-

2. Security – Encryption & Authentication

- Enable **TLS encryption** on the TCP server.
- Use **self-signed certificates** (server-side, trusted by the client).
- Require clients to authenticate using a **token** (first message after connecting must be `AUTH <token>`).
- Implement **rate limiting** (e.g., max 5 messages per 10 seconds per client).
- Validate inputs:
 - Reject oversized payloads (>4KB).
 - Reject malformed or binary data in text-based protocols.

Acceptance Criteria:

- Client refuses to connect if TLS handshake fails.
 - Server closes connection on invalid/missing authentication.
 - Rate limiting enforced per client.
 - Logs security events (failed auth, rate-limit, invalid input) without exposing sensitive data.
-

3. Linux Integration – Deployment & Security Practices

- Provide a **systemd service file** (or shell script) to run the server as a background service.
- Configure logs to be written under `/var/log/myapp/` with correct permissions (`600`).
- Ensure secrets (auth tokens, TLS certs) are read from **environment variables** or **config files** with restricted permissions.
- Implement a Linux shell script `secure_check.sh` that:
 - Verifies the server port is open using `ss` or `netstat`.
 - Checks if the server process is running.

- Tests file permissions for certs/config/logs.

Acceptance Criteria:

- Server can be started/stopped via `systemctl` or provided shell script.
 - Logs are available under `/var/log/myapp/` and secured with correct permissions.
 - `secure_check.sh` outputs PASS/FAIL for each security check.
-

Method of Submission

1. Codebase

- Submit Python source code (`server.py`, `client.py`).
- Include configuration files (`config.json` or `.env`).
- Provide TLS certificate and key generation steps (`README.md` with commands).

2. Linux Artifacts

- Submit your `systemd` unit file (`myapp.service`) or startup shell script.
- Submit `secure_check.sh` script.

3. Documentation

- A `README.md` explaining:
 - How to set up and run the server/client.
 - How to generate TLS certificates.
 - How authentication works.
 - How to run `secure_check.sh`.

4. Submission Format

- Package everything into a `.tar.gz` or `.zip` archive.
- Include a `run.sh` script that launches the server with one command.

5. Design architecture in .pdf